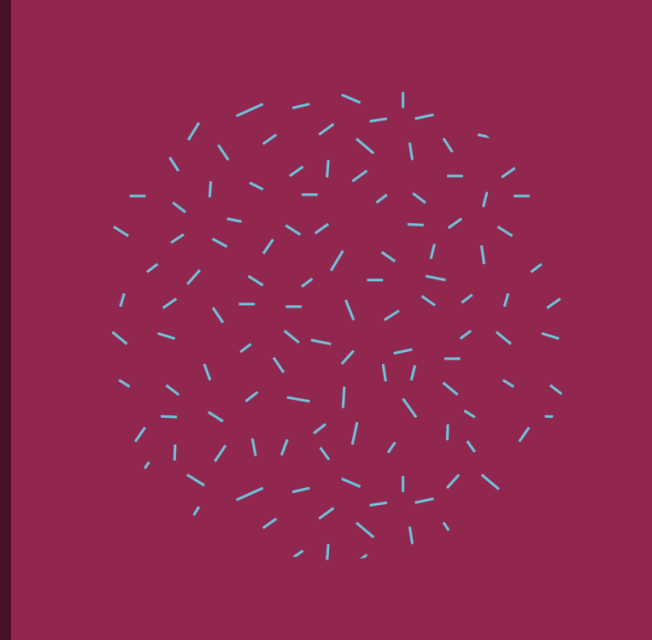
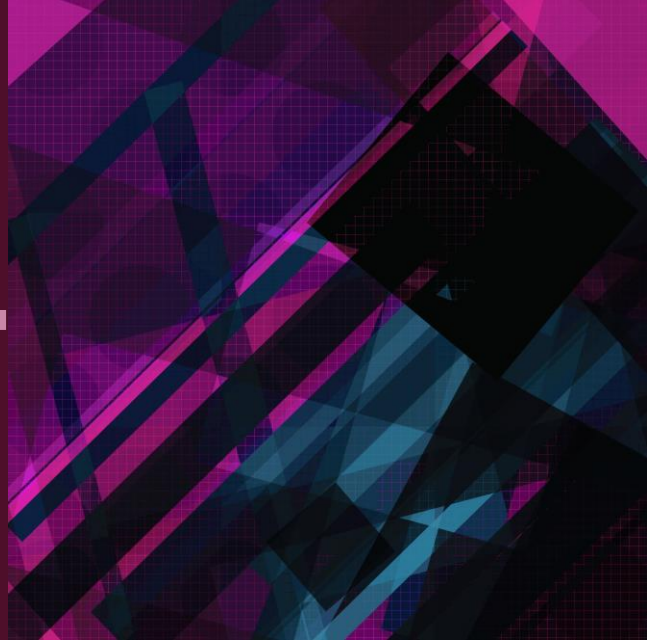




Data Analysis

Regression

Section 5

Eng. Asmaa Fathy

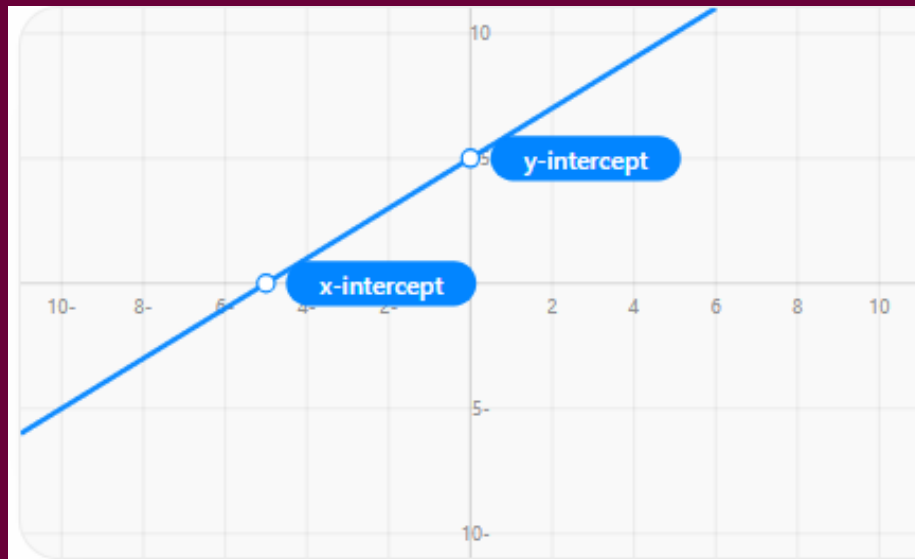
Regression

- Regression is a supervised learning task in which an algorithm learns to predict a continuous output value (or target) based on input features. The goal of regression is to find the best model that accurately predicts the target value for new, unseen inputs.
- SIMPLE REGRESSION
- MULTIPLE REGRESSION
- REGULARIZATION
- CROSS-VALIDATION
- ENSEMBLE METHODS



SIMPLE REGRESSION

- It's a technique we use to find a straight-line relationship between two variables:
- **Independent Variable (X) → Cause or Input**
- **Dependent Variable (Y) → Outcome or Result**
- Simple Example: Number of Study Hours (X) → Grade (Y)
- Basic Concept: We try to draw a straight line that represents the relationship:



$$y = mx + b$$

m = Slope (How much Y changes as X increases)

b = Starting Point (When $X = 0$)

Use this line to predict new values and understand the relationship between the variables

California Housing Price Using Simple Regression

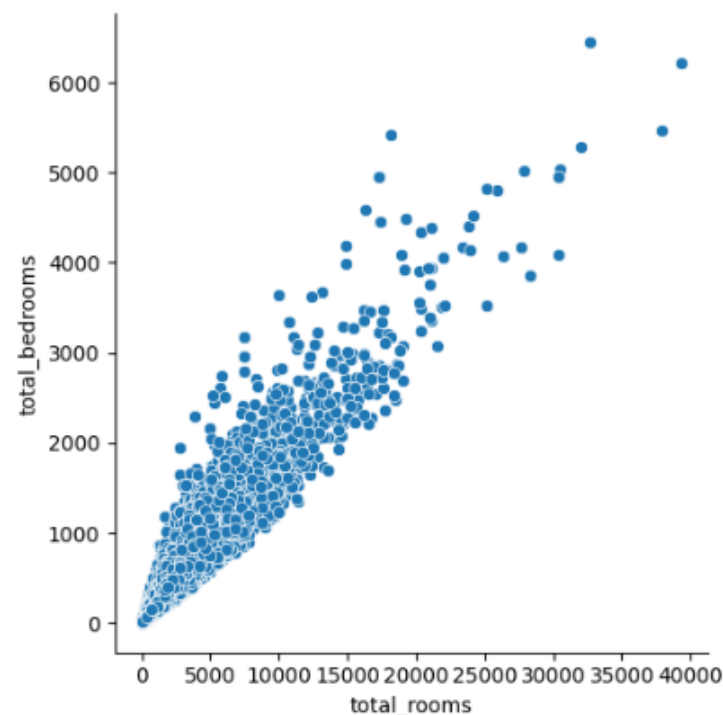
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('C:/Users/Admin/Documents/Data Mining Practical/housing2.csv')
df.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

```
# A Scatter Plot of Total Rooms VS Total Bedrooms
sns.relplot(data=df,x='total_rooms',y='total_bedrooms')
# we're seeing: if the number of rooms increases, does the number of bedrooms also increase?
```

<seaborn.axisgrid.FacetGrid at 0x27bd7eab230>



```
# Prepare independent
x = np.array(df['total_rooms'])
```

```
# Prepare dependent
y = np.array(df['total_bedrooms'])
```

```
# Checking for missing values
print(np.isnan(y).sum())
```

207

```
# Cleaning up the data (removing missing values)
mask = ~np.isnan(y)
```

```
# We use the mask to remove rows containing NaN values.
# It's very important to keep X and Y in the same rows.
x = x[mask]
y = y[mask]
```

```
# We convert them to a 2D shape.
# `-1` → Let NumPy automatically determine the number of rows (Number of data).
# `1` → One column (one feature (total_rooms))
```

```
x = x.reshape(-1, 1)
y = y.reshape(-1, 1)
```

```
print(x.shape)
print(y.shape)
```

```
(20433, 1)
(20433, 1)
```

```
# Split train and test data
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.20)
```

```
# Train the model
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
model = reg.fit(x_train, y_train)
```

```
model.coef_, model.intercept_
```

```
#  $y = m*x + b$ 
# `coef_` → slope ( $m$ )
# `intercept_` → starting point ( $b$ )
```

```
(array([[0.1800464]]), array([63.88746613]))
```

```

# Evaluate the model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# The model predicts new values
y_pred = model.predict(x_test)

# Average difference between actual and predicted values
mae = mean_absolute_error(y_test, y_pred)

# Same concept, but magnifies large errors
mse = mean_squared_error(y_test, y_pred)

# The square root of the MSE
rmse = np.sqrt(mse)

# How well the model understands the data (from 0 to 1)
r2 = r2_score(y_test, y_pred)
# if r2 Close to 1 → Good
# if r2 Close to 0 → Bad

print('MAE:', mae)
print('MSE:', mse)
print('RMSE:', rmse)
print('R2:', r2)

```

```

MAE: 97.8320427433787
MSE: 22297.385877992816
RMSE: 149.32309224628594
R2: 0.8618342042254601

```

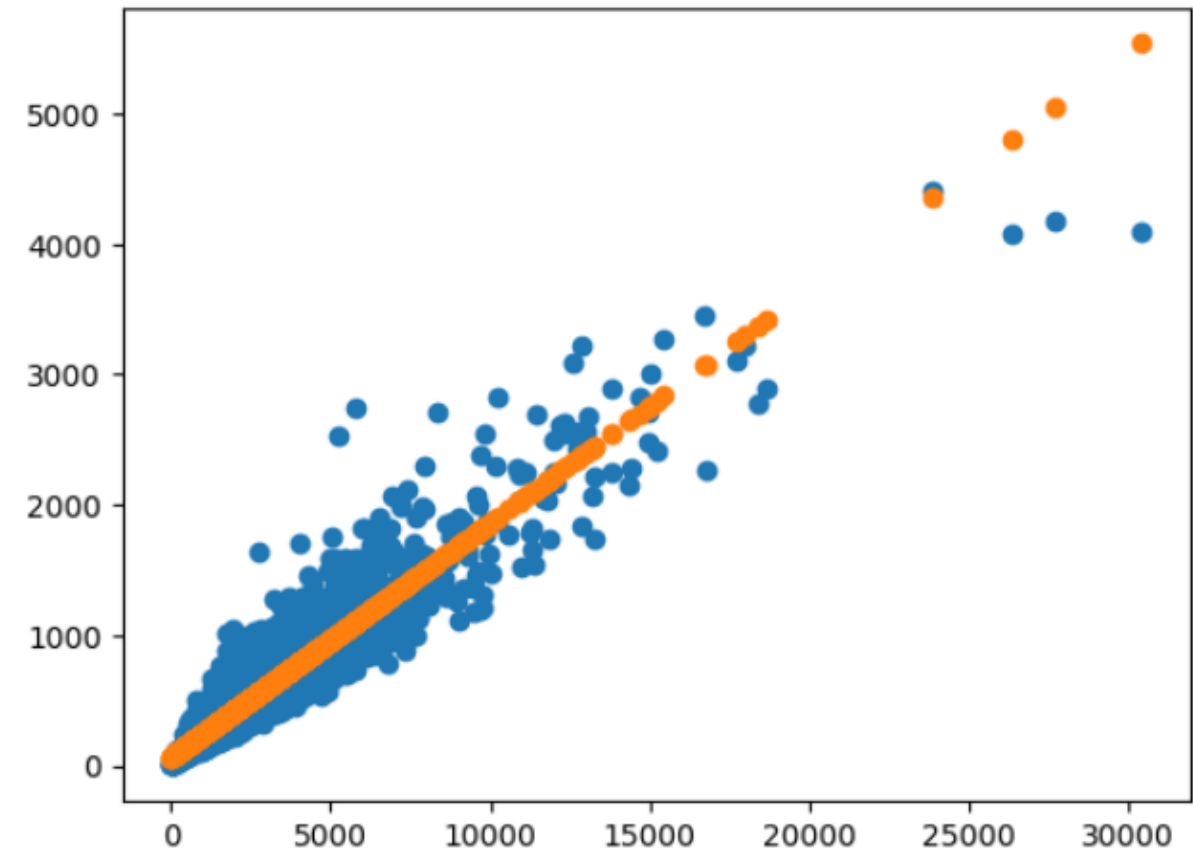
```

# Visualize your result
plt.scatter(x_test, y_test) # Actual scores
plt.scatter(x_test, y_pred) # Score predicted by the model

```

This graph:
 # First scatter → Actual score
 # Second Scatter → Prediction
 # If the two points are close together → The model is working well
 # If they are far apart → The model is weak

<matplotlib.collections.PathCollection at 0x27bdb95c190>



Try Simple Linear Regression Again

```
# Prepare independent variable
x = np.random.uniform(-3, 3, (100)).reshape(-1,1)

# We create *100 random numbers* between -3 and 3.
# `.reshape(-1,1)` to make it 2D.

# Prepare dependent variable

y = x * x - x - 1
# This is the actual equation for the data:  $y = x^2 - x - 1$ 
# This is a curve (not a straight line) relationship

error = np.random.rand((100)).reshape(-1, 1)
# We add **random noise to make the data more realistic (not perfect)

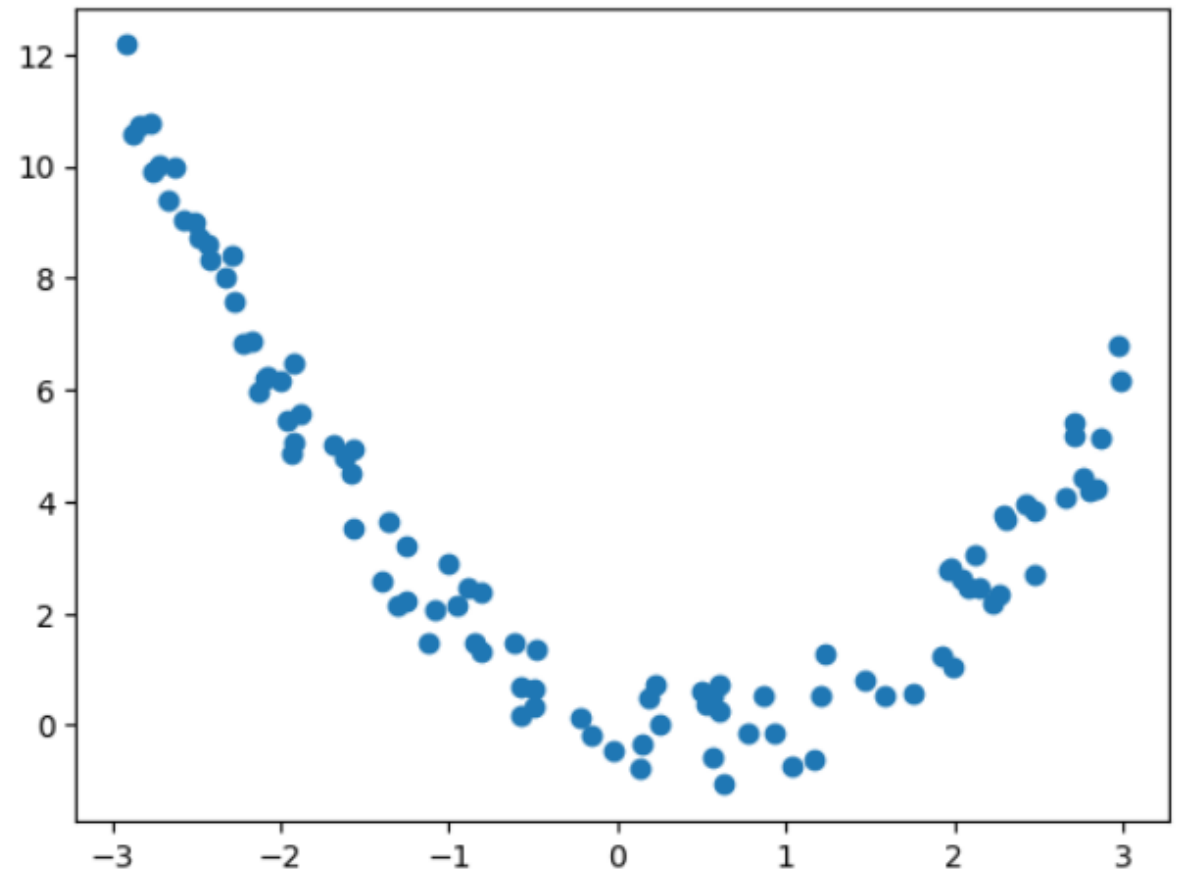
y = y + error*2
# We increase the noise on y to make it more realistic

y = y.reshape(-1, 1)
# We check that y is the correct shape 2D
```

```
plt.scatter(x, y)
```

Plots points to see the relationship
You should see a curve, not a line

<matplotlib.collections.PathCollection at 0x27bddb17110>



```
# Split train and test data
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.20)
```

```
# Train the model
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
model = reg.fit(x_train, y_train)
```

```
model.coef_, model.intercept_
```

```
(array([[ -1.12457977]]), array([ 3.42818417]))
```

```
# Evaluate the model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
y_pred = model.predict(x_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print('MAE:', mae)
print('MSE:', mse)
print('RMSE:', rmse)
print('R2:', r2)
```

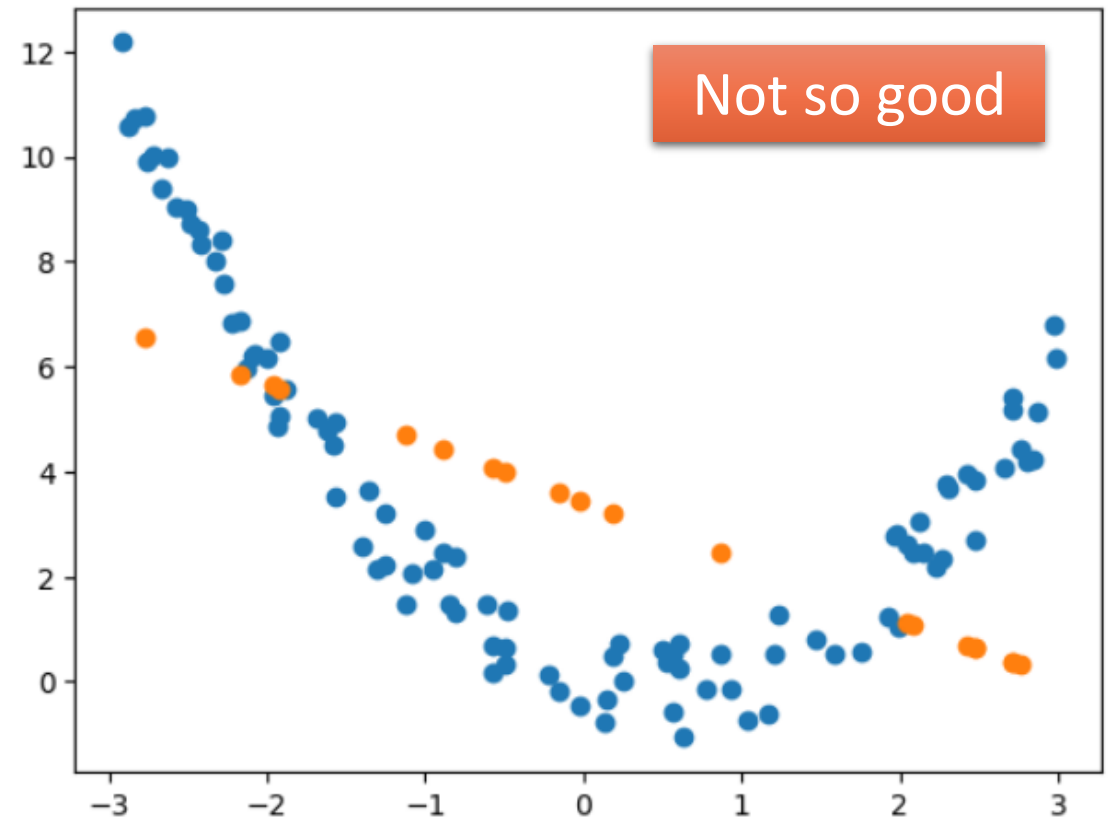
```
MAE: 2.8164347589394234
MSE: 9.851418024714523
RMSE: 3.138696867286569
R2: -0.29583397153800717
```

$R^2 \approx -0.29$ -> Very bad

- **Why is the result bad?** Because The data is a **curve** and the model is a **straight line**

```
# Visualize your result
plt.scatter(x, y) # Actual -> curve
plt.scatter(x_test, y_pred) # Predicted -> line
```

<matplotlib.collections.PathCollection at 0x27bddc08f50>



Solution: Polynomial Regression

The basic idea :

- We have: `x` → only one number -> This means the model sees only one feature
 - But we want it to understand a curve, not a line.

What do we do?

- We transform `x` into a larger shape, Instead of being $x \rightarrow$ It becomes: $[1, x, x^2]$
- Examples:

x	After Conversion
1	[1, 1, 1]
2	[1, 2, 4]
3	[1, 3, 9]

- Why do we do this? To make the model learn this equation: $y = ax^2 + bx + c$ Instead of $y = mx + b$

Try Polynomial Features

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.20)
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
# Use up to  $x^2$ 
# Meaning:  $x$  and  $x^2$ 
```

```
poly = PolynomialFeatures(degree=2)
```

```
# It does 3 things:
# 1. Learns the data shape (fit)
# 2. Transforms it (transform)
# 3. Gives you the new shape
# Output: [1, x, x^2]
```

```
poly_train = poly.fit_transform(x_train.reshape(-1, 1))
```

```
# Apply the same transformation to test
poly_test = poly.transform(x_test.reshape(-1, 1))
```

poly_train

```
array([[ 1.00000000e+00, -1.92923701e+00,  3.72195543e+00],
       [ 1.00000000e+00, -5.80000168e-01,  3.36400194e-01],
       [ 1.00000000e+00, -1.25383502e+00,  1.57210226e+00],
       [ 1.00000000e+00, -1.57910493e+00,  2.49357238e+00],
       [ 1.00000000e+00, -8.11495244e-01,  6.58524531e-01],
       [ 1.00000000e+00, -2.13333844e+00,  4.55113290e+00],
       [ 1.00000000e+00,  1.82574477e-01,  3.33334397e-02],
       [ 1.00000000e+00, -1.87834243e+00,  3.52817028e+00],
       [ 1.00000000e+00,  1.98733012e+00,  3.94948100e+00],
       [ 1.00000000e+00,  2.70982253e+00,  7.34313815e+00],
       [ 1.00000000e+00, -1.57013071e+00,  2.46531045e+00],
       [ 1.00000000e+00, -1.08652844e+00,  1.18054404e+00],
       [ 1.00000000e+00,  2.48186807e-01,  6.15966913e-02],
       [ 1.00000000e+00, -1.25290558e+00,  1.56977238e+00],
       [ 1.00000000e+00, -1.51982984e-01,  2.30988275e-02],
       [ 1.00000000e+00,  1.16428365e+00,  1.35555642e+00])
```

```
# Train the model with polynomial features
from sklearn.linear_model import LinearRegression
poly_reg_model = LinearRegression()
poly_reg_model.fit(poly_train, y_train)
```

LinearRegression

Parameters

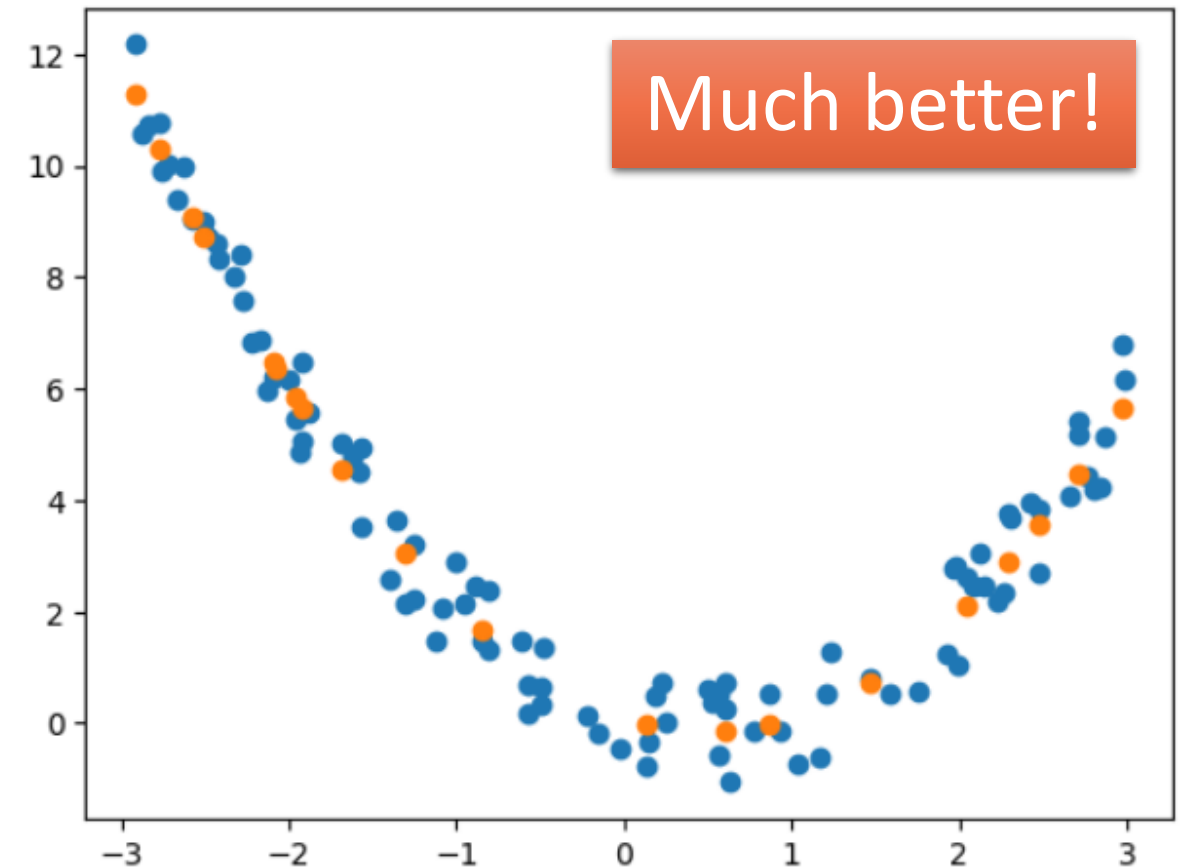
```
# Evaluate the model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
y_pred = poly_reg_model.predict(poly_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

```
print('MAE:', mae)
print('MSE:', mse)
print('RMSE:', rmse)
print('R2:', r2)
```

```
MAE: 0.5614020863614699
MSE: 0.4164662684952248
RMSE: 0.6453419779428771
R2: 0.966284781628296
```

```
# Visualize the model
plt.scatter(x, y)
plt.scatter(x_test, y_pred)
```

<matplotlib.collections.PathCollection at 0x27bdf3d2710>



Is the degree
higher, the result
better? How
about we do a
degree as 30?

```
poly = PolynomialFeatures(degree=30)

poly_train = poly.fit_transform(x_train.reshape(-1, 1))
poly_test = poly.transform(x_test.reshape(-1, 1))

poly_reg_model = LinearRegression()
poly_reg_model.fit(poly_train, y_train)

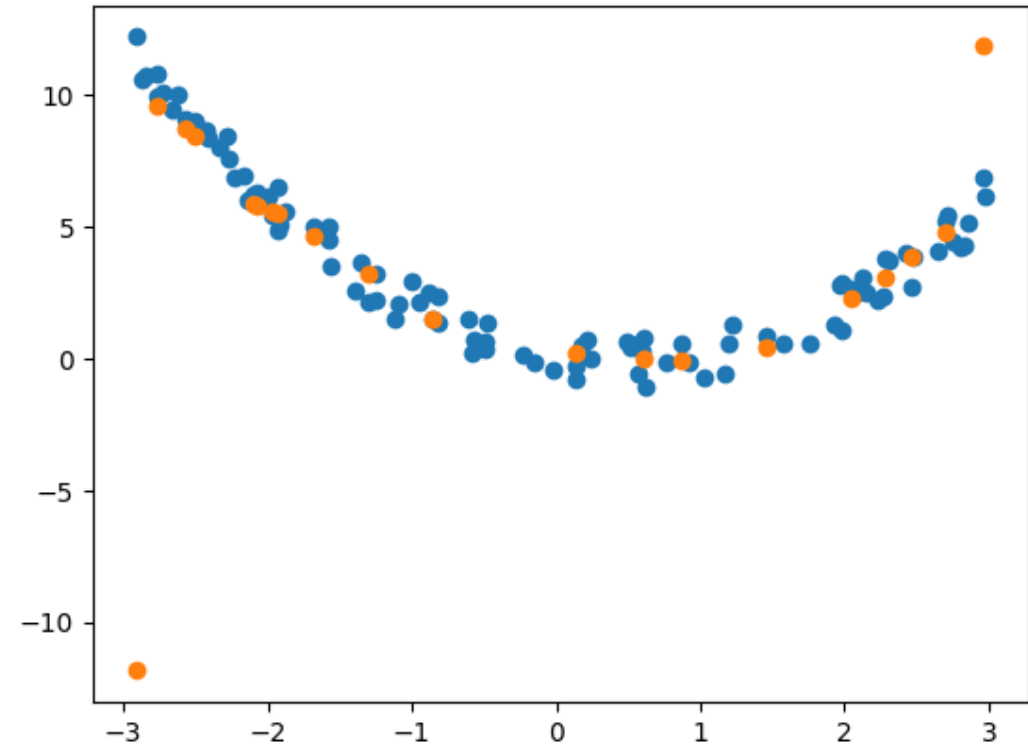
y_pred = poly_reg_model.predict(poly_train)
mae = mean_absolute_error(y_train, y_pred)
mse = mean_squared_error(y_train, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_train, y_pred)
print('Training:
MAE: {}
MSE: {}
RMSE: {}
R^2 {}'.format(mae, mse, rmse, r2))

y_pred = poly_reg_model.predict(poly_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print('Testing:
MAE: {}
MSE: {}
RMSE: {}
R^2 {}'.format(mae, mse, rmse, r2))
```

```
Training:
MAE: 0.4340489565857874
MSE: 0.2753240780633635
RMSE: 0.5247133294127028
R^2 0.9723344183993209
Testing:
MAE: 1.9934405356018854
MSE: 30.53693370246807
RMSE: 5.526023317220808
R^2 -1.472131517159743
```

```
plt.scatter(x, y)
plt.scatter(x_test, y_pred)
```

<matplotlib.collections.PathCollection at 0x27bdf49de50>



Nope!
High-degree Polynomial Regression may
result in overfitting
Training result is better,
but testing result is poor!

MULTIPLE REGRESSION

What does the equation look like?

Instead: $y = mx + b$.

Remains: $y = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n$

$(x_1, x_2, x_3) \rightarrow$ Inputs

$(b_1, b_2) \rightarrow$ Effect of each

$(b_0) \rightarrow$ Intercept

This means each feature has a weight.

- It's the same as Linear Regression, but instead of having one variable (X), you now have more than one variable.
- Imagine it like this:
- Simple Regression: "I'm only looking at one thing."
- Multiple Regression: "I'm looking at all the factors together."
- **Simple Example:** We want to predict the price of a house. Instead of relying on just one thing, like Area only, we use: Area, Number of rooms, Location, and Income. All of these are called Features ($X_1, X_2, X_3, \dots$)

Very Important Things in Multiple Regression:

1. **Feature Selection:** Not just any feature will do. You must choose only the useful ones.
2. **Multicollinearity:** When features are similar to each other. For example, total_rooms and total_bedrooms are very related. This can sometimes ruin the model.
3. **Scaling (Sometimes):** If the numbers are large and small together, we perform normalization or standardization.

The Main Idea: The model tries to understand how each factor affects based on the result.

For example: Each additional room = +5000 EGP , Each higher income = +20000 EGP

Try Dependent as Median House Value, and two Independent Variables

```
# Prepare independent
X = np.array(df[['total_rooms', 'median_income']])
```

```
# Prepare dependent
Y = np.array(df['total_bedrooms'])
```

```
# Checking for missing values
print(np.isnan(Y).sum())
```

207

```
# Cleaning up the data (removing missing values)
mask = ~np.isnan(Y)
X = X[mask]
Y = Y[mask]
```

```
X = X.reshape(-1, 2)
Y = Y.reshape(-1, 1)
```

```
print(X.shape)
print(Y.shape)
```

```
(20433, 2)
(20433, 1)
```

```
# Split train and test data
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20)
```

```
# Train the model
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
model = reg.fit(X_train, y_train)
```

```
model.coef_, model.intercept_
```

```
(array([[ 0.188022 , -44.1855963]]), array([213.38402642]))
```

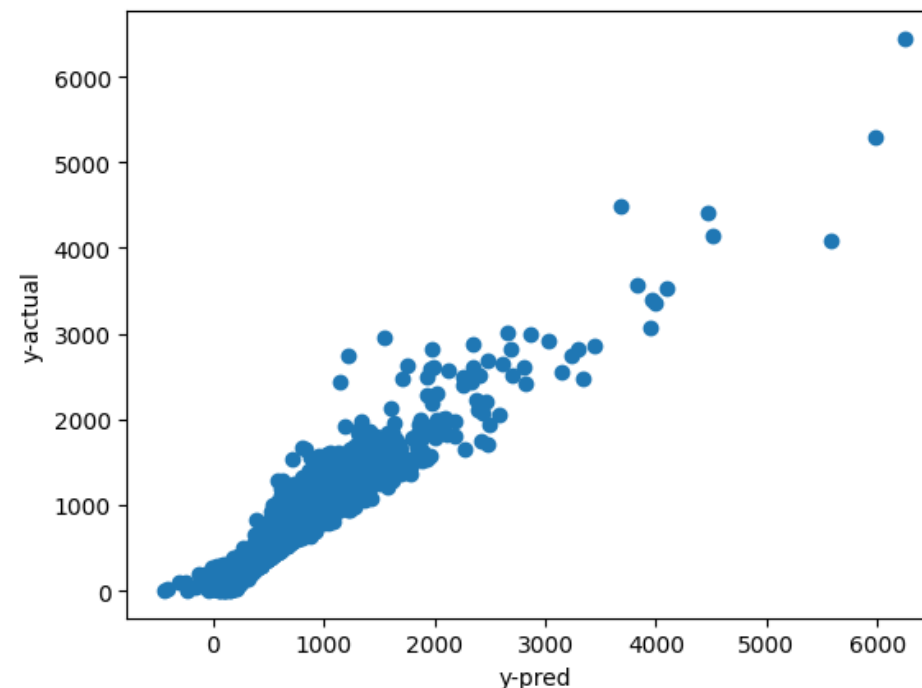
```
# Evaluate the model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

```
print('MAE: {} MSE: {} RMSE: {} R^2 {}'.format(mae, mse, rmse, r2))
```

MAE: 81.54801815986218 MSE: 17203.65724323212 RMSE: 131.16271285404292 R^2 0.9065166566612356

```
# Visualize your result
plt.scatter(y_pred, y_test)
plt.xlabel('y-pred')
plt.ylabel('y-actual')
```

Text(0, 0.5, 'y-actual')



Remember when we did: Polynomial score = 30.
The model was very good at training and failed at testing.
This is called Overfitting: the model becomes memorizing data instead of understanding it.

REGULARIZATION

- A method to prevent the model from memorizing excessive data.
- A technique used to reduce overfitting
- It adds a penalty to the model to make it simpler

Instead of minimizing only: Error
We minimize: Error + Penalty
The model is punished if it becomes too complex
$$Y = w_1x_1 + w_2x_2 + b$$

The w represents the effect of each feature
If it's very large, the model becomes complex

Types of REGULARIZATION

Ridge (L2)	Lasso (L1)	Elastic Net
$Loss = Error + \lambda \sum w^2$	$Loss = Error + \lambda \sum w $	$Loss = Error + \lambda_1 \sum w + \lambda_2 \sum w^2$
▪ Reduces large coefficients. ▪ Keeps all features but makes their values smaller	▪ Reduces coefficients and can make some of them = 0 ▪ Removes unimportant features ▪ Works like feature selection	▪ Combines Ridge (L2) and Lasso (L1). ▪ Penalizes large values (like Ridge) and can remove features (like Lasso).

The most important parameters

Alpha (λ)

- Controls the strength of regularization
- Small alpha $\rightarrow$ weak penalty $\rightarrow$ possible overfitting
- Large alpha $\rightarrow$ strong penalty $\rightarrow$ simpler model

l1_ratio

- Determines the ratio between Lasso and Ridge.
- If Value 0 $\rightarrow$ Ridge only
- If Value 1 $\rightarrow$ Lasso only
- If Value 0.5 $\rightarrow$ Both together

Step-by-step Python tutorial for regression analysis using a dataset, demonstrating Linear Regression, Polynomial Regression, and regularization techniques (Ridge, Lasso, Elastic Net) to handle overfitting

- You can either use a real dataset or generate a dummy dataset for this tutorial. For simplicity, let's create a dummy dataset using NumPy.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score, mean_squared_error
```

Generating a dummy dataset

Give 100 numbers starting from 0 up to 5 with equal spaces between them
`X = np.linspace(0, 5, 100).reshape(-1, 1)`

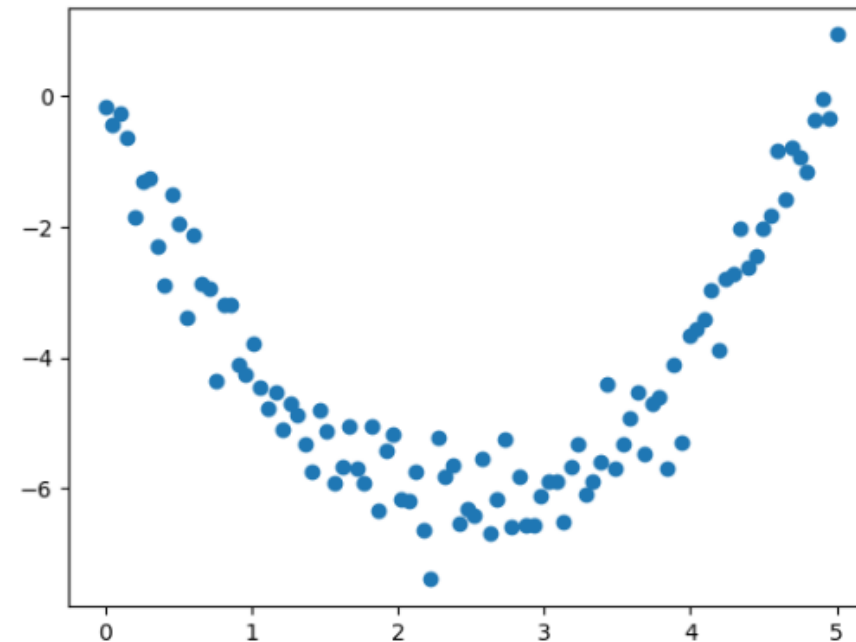
*# $-5 * X$ --> This is a linear part. As X increases $\rightarrow$ the value decreases*
X^2 --> This is the square of X . It makes the relationship a curve
Together, the equation becomes: $y = x^2 - 5x$. This means a curve, not a straight line.
`np.random.normal(0, 0.5, X.shape[0])` --> This creates noise to make the data more realistic. Random numbers (Average
$y = (\text{curve}) + (\text{noise})$

`y = -5*X + X**2 + np.random.normal(0, 0.5, X.shape[0]).reshape(-1, 1)`

0 $\rightarrow$ mean
0.5 $\rightarrow$ standard deviation

```
plt.scatter(X, y)
```

<matplotlib.collections.PathCollection at 0x2787da46660>



Linear Regression

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Create and fit the linear regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Make predictions on training and testing data
y_train_pred = linear_model.predict(X_train)
y_test_pred = linear_model.predict(X_test)

# Calculate R-squared and mean squared error for evaluation
linear_r2_train = r2_score(y_train, y_train_pred)
linear_r2_test = r2_score(y_test, y_test_pred)
linear_mse_train = mean_squared_error(y_train, y_train_pred)
linear_mse_test = mean_squared_error(y_test, y_test_pred)

print(f"Linear Regression:")
print(f"Training R-squared: {linear_r2_train:.4f} , Training MSE: {linear_mse_train:.4f}")
print(f"Testing R-squared: {linear_r2_test:.4f} , Testing MSE: {linear_mse_test:.4f}")
```

Linear Regression:

Training R-squared: 0.0042 , Training MSE: 4.0681

Testing R-squared: -0.0561 , Testing MSE: 3.6412

Too simple (Underfitting)

The model failed
because the data is
originally: $y = x^2 - 5x$
→ A curve and the
model is a straight line

Polynomial Regression with Degree of 2

```
# Transform the features to include polynomial features of degree 2
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

# Split the polynomial features into training and testing sets
X_poly_train, X_poly_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.2, random_state=0)

# Create and fit the polynomial regression model
poly_model = LinearRegression()
poly_model.fit(X_poly_train, y_train)

# Make predictions on training and testing data
y_train_pred_poly = poly_model.predict(X_poly_train)
y_test_pred_poly = poly_model.predict(X_poly_test)

# Calculate R-squared and mean squared error for evaluation
poly_r2_train = r2_score(y_train, y_train_pred_poly)
poly_r2_test = r2_score(y_test, y_test_pred_poly)
poly_mse_train = mean_squared_error(y_train, y_train_pred_poly)
poly_mse_test = mean_squared_error(y_test, y_test_pred_poly)

print(f"\nPolynomial Regression (Degree 2):")
print(f"Training R-squared: {poly_r2_train:.4f} , Training MSE: {poly_mse_train:.4f}")
print(f"Testing R-squared: {poly_r2_test:.4f} , Testing MSE: {poly_mse_test:.4f}")
```

Polynomial Regression (Degree 2):
Training R-squared: 0.9458 , Training MSE: 0.2214
Testing R-squared: 0.9105 , Testing MSE: 0.3084

The model is now
excellent because he
understood the curve

Very suitable

Polynomial Regression with Higher Degree

```
# Transform the features to include polynomial features of higher
degree = 21
poly_high_degree = PolynomialFeatures(degree=degree)
X_poly_high_degree = poly_high_degree.fit_transform(X)

# Split the high-degree polynomial features into training and testing sets
X_poly_high_degree_train, X_poly_high_degree_test, y_train, y_test= train_test_split(X_poly_high_degree, y, test_size=0.2, random_state=0)

# Create and fit the high-degree polynomial regression model
poly_model_high_degree = LinearRegression()
poly_model_high_degree.fit(X_poly_high_degree_train, y_train)

# Make predictions on training and testing data
y_train_pred_high_degree= poly_model_high_degree.predict(X_poly_high_degree_train)
y_test_pred_high_degree= poly_model_high_degree.predict(X_poly_high_degree_test)

# Calculate R-squared and mean squared error for evaluation
poly_r2_train_high_degree= r2_score(y_train, y_train_pred_high_degree)
poly_r2_test_high_degree= r2_score(y_test, y_test_pred_high_degree)
poly_mse_train_high_degree= mean_squared_error(y_train, y_train_pred_high_degree)
poly_mse_test_high_degree= mean_squared_error(y_test, y_test_pred_high_degree)

print(f"\nPolynomial Regression (Degree {degree}):")
print(f"Training R-squared: {poly_r2_train_high_degree:.4f} , Training MSE: {poly_mse_train_high_degree:.4f}")
print(f"Testing R-squared: {poly_r2_test_high_degree:.4f} , Testing MSE: {poly_mse_test_high_degree:.4f}")
```

Performance decreased
because the model
became too complex →
It started overfitting

Polynomial Regression (Degree 21):
Training R-squared: 0.8359 , Training MSE: 0.6704
Testing R-squared: 0.7635 , Testing MSE: 0.8153

Overfitting

Regularization (Ridge, Lasso, Elastic Net)

Controlling Complexity

```
# Regularization strengths
alpha_ridge = 0.001
alpha_lasso = 0.001
alpha_elasticnet = 0.001
l1_ratio_elasticnet = 0.5

# Create and fit the Ridge, Lasso, and ElasticNet regression models
ridge_model = Ridge(alpha=alpha_ridge)
ridge_model.fit(X_poly_high_degree_train, y_train)

lasso_model = Lasso(alpha=alpha_lasso)
lasso_model.fit(X_poly_high_degree_train, y_train)

elasticnet_model = ElasticNet(alpha=alpha_elasticnet, l1_ratio=l1_ratio_elasticnet)
elasticnet_model.fit(X_poly_high_degree_train, y_train)

# Make predictions on training and testing data for all regularized models
y_train_pred_ridge = ridge_model.predict(X_poly_high_degree_train)
y_test_pred_ridge = ridge_model.predict(X_poly_high_degree_test)

y_train_pred_lasso = lasso_model.predict(X_poly_high_degree_train)
y_test_pred_lasso = lasso_model.predict(X_poly_high_degree_test)

y_train_pred_elasticnet = elasticnet_model.predict(X_poly_high_degree_train)
y_test_pred_elasticnet = elasticnet_model.predict(X_poly_high_degree_test)

# Calculate R-squared and mean squared error for evaluation
ridge_r2_train = r2_score(y_train, y_train_pred_ridge)
ridge_r2_test = r2_score(y_test, y_test_pred_ridge)
ridge_mse_train = mean_squared_error(y_train, y_train_pred_ridge)
ridge_mse_test = mean_squared_error(y_test, y_test_pred_ridge)

lasso_r2_train = r2_score(y_train, y_train_pred_lasso)
lasso_r2_test = r2_score(y_test, y_test_pred_lasso)
lasso_mse_train = mean_squared_error(y_train, y_train_pred_lasso)
lasso_mse_test = mean_squared_error(y_test, y_test_pred_lasso)

elasticnet_r2_train = r2_score(y_train, y_train_pred_elasticnet)
elasticnet_r2_test = r2_score(y_test, y_test_pred_elasticnet)
elasticnet_mse_train = mean_squared_error(y_train, y_train_pred_elasticnet)
elasticnet_mse_test = mean_squared_error(y_test, y_test_pred_elasticnet)

print("\nRegularization:")
print(f"Ridge Regression- Training R-squared: {ridge_r2_train:.4f} , Testing R-squared: {ridge_r2_test:.4f}")
print(f"Lasso Regression- Training R-squared: {lasso_r2_train:.4f} , Testing R-squared: {lasso_r2_test:.4f}")
print(f"ElasticNet Regression- Training R-squared: {elasticnet_r2_train:.4f} , Testing R-squared: {elasticnet_r2_test:.4f}")
```

Regularization:

Ridge Regression- Training R-squared: 0.9499 , Testing R-squared: 0.9098

Lasso Regression- Training R-squared: 0.9464 , Testing R-squared: 0.9132

ElasticNet Regression- Training R-squared: 0.9462 , Testing R-squared: 0.9131

Performance improved and
stabilized because the
model is now less complex.
It no longer memorize data.

CROSS-VALIDATION

The First Problem we were dividing the data into Train and Test
The result might depend on this division only. Sometimes the model turns out well. Sometimes it turns out badly
The Solution: Cross-validation → Instead of dividing it once, we divide it multiple times.

- We test the model on several divisions, not just one.
- We divide the data into parts (folds).
- **For example: 5 Parts** → We use 4 training parts and One test part and repeat the process 5 times. Each time we change the part we test on.
- In the end, we calculate the average performance: **Average R^2 or Average Accuracy.**
- Benefits: More accurate evaluation, reduces overfitting, and doesn't rely on a single partition.

Time	Train	Test
1	1,2,3,4	5
2	1,2,3,5	4
3	1,2,4,5	3
4	1,3,4,5	2
5	2,3,4,5	1

Types of CROSS-VALIDATION

K-Fold (Regular)	Stratified K-Fold
We divide the data into equal parts without even looking at them.	We divide a little more intelligently maintain the same ratio.
Example: You have data with 10 elements: [A, A, A, A, A, B, B, B, B, B] (Meaning 5 from A and 5 from B). We divided it into 5 parts: Fold1: A, A Fold2: A, A Fold3: A, B Fold4: B, B Fold5: B, B The problem? Some folds only contain A or only B The model might learn on only one type or get an incorrect evaluation.	Example: [A, A, A, A, A, B, B, B, B, B]. Each fold should contain: A + B. For example: Fold1: A, B Fold2: A, B Fold3: A, B Fold4: A, B Fold5: A, B
▪ Random division ▪ May contain bias ▪ Suitable for Regression	▪ Proportional division ▪ More accurate ▪ Best for classification

We will generate a dummy dataset for regression and demonstrate how to perform 3-fold, 5-fold, 10-fold, and leave-one-out cross-validation for regression models. Finally, we will compare the results to understand the performance of each approach.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold, LeaveOneOut
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Generating a dummy dataset

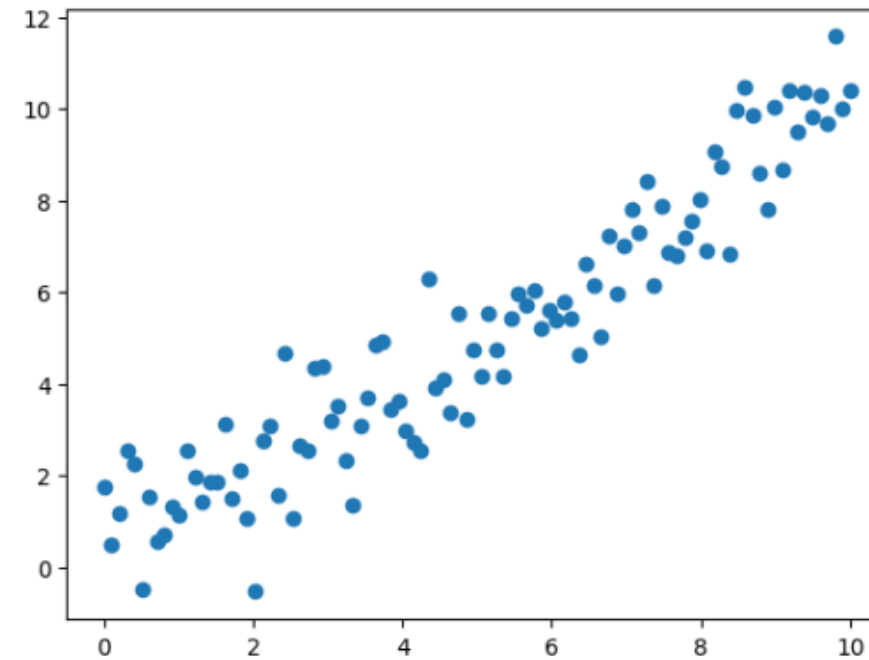
# Makes random numbers yield the same result every time
np.random.seed(0)

# We generate 100 numbers from 0 to 10 with equal spacing between them
X = np.linspace(0, 10, 100).reshape(-1, 1)

# y = X --> The basic relationship: A straight line
# np.random.normal(0, 1, X.shape[0]) --> This is noise (Average = 0, Intensity = 1), It makes the data not perfect
y = X + np.random.normal(0, 1, X.shape[0]).reshape(-1, 1)

plt.scatter(X, y)
```

<matplotlib.collections.PathCollection at 0x2780396c550>



Cross-validation using different techniques and compare the results

```
# Create a List to store the MSEs for different techniques
# We'll put the result of each Cross Validation method in it
# We'll sum the results of each method and compare them at the end
mse_scores = []

# List of cross-validation techniques to be used
cv_methods = ['3-fold', '5-fold', '10-fold', 'Leave-One-Out']

for cv_method in cv_methods:
    if cv_method == '3-fold':
        cv = KFold(n_splits=3, shuffle=True, random_state=0)
    elif cv_method == '5-fold':
        cv = KFold(n_splits=5, shuffle=True, random_state=0)
    elif cv_method == '10-fold':
        cv = KFold(n_splits=10, shuffle=True, random_state=0)
    else:
        cv = LeaveOneOut()

    # Leave-One-Out: Each time, we only use one point as a test, and the rest train. Means maximum possible division

    # Create and fit the Linear regression model using cross-validation
    model = LinearRegression()

    # List to Save MSE for Each Fold
    mse_scores_cv = []

    # Splitting and Training
    for train_idx, test_idx in cv.split(X):
        # We actually divide the data
        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        # The model learns only from the training
        model.fit(X_train, y_train)

        # It tests on testing
        y_pred = model.predict(X_test)

        # We measure how much the error is.
        mse = mean_squared_error(y_test, y_pred)
        # We save each result separately.
        mse_scores_cv.append(mse)

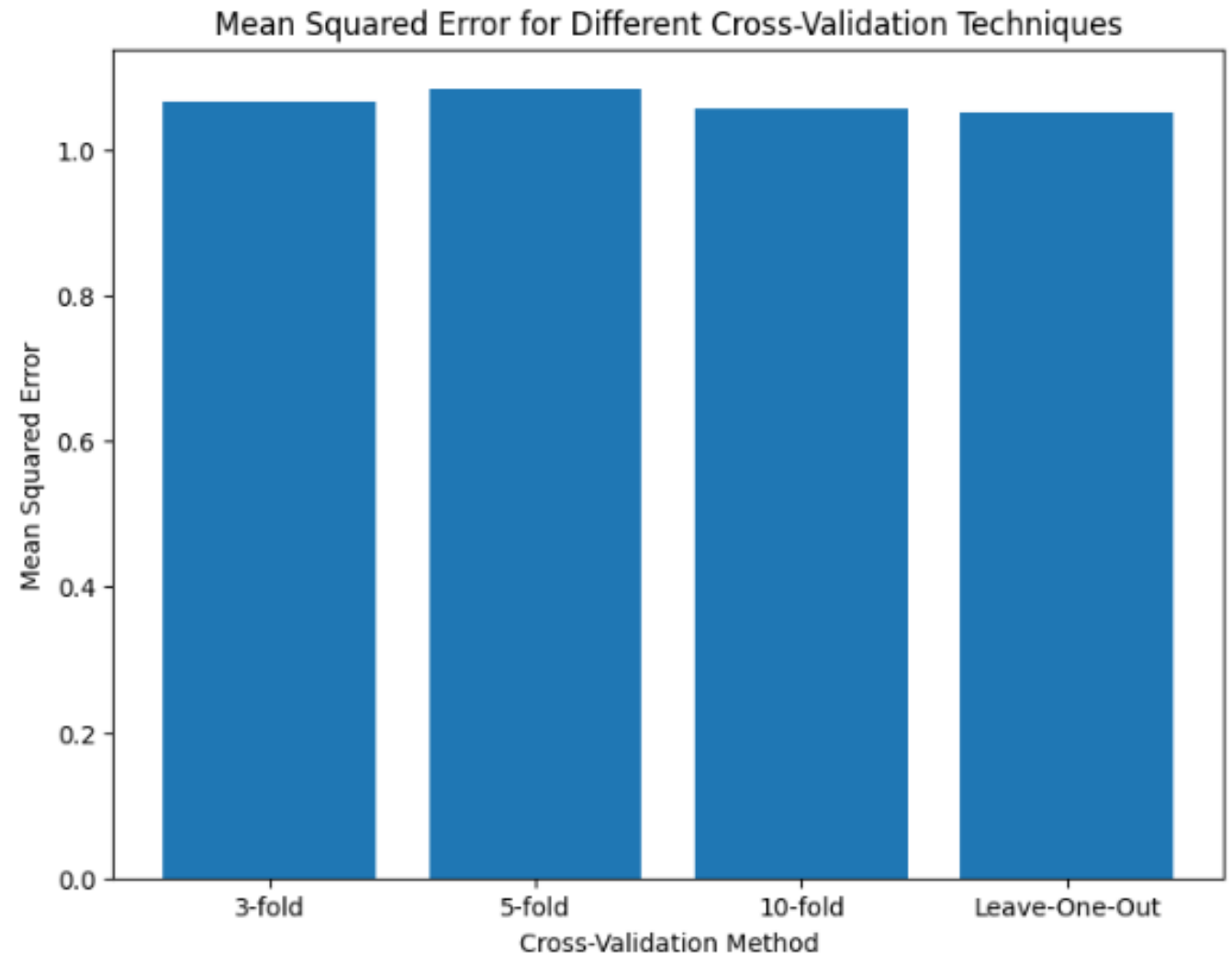
    # We find the average performance for the entire method.
    mse_scores.append(np.mean(mse_scores_cv))

# Display the mean squared errors for different cross-validation techniques
for i, cv_method in enumerate(cv_methods):
    print(f"{cv_method} Cross-Validation- Mean Squared Error: {mse_scores[i]:.4f}")
```

3-fold Cross-Validation- Mean Squared Error: 1.0650
5-fold Cross-Validation- Mean Squared Error: 1.0840
10-fold Cross-Validation- Mean Squared Error: 1.0566
Leave-One-Out Cross-Validation- Mean Squared Error: 1.0510

Visualize the mean squared errors for different cross-validation techniques using a bar plot.

```
# Plotting mean squared errors for different cross-validation techniques
plt.figure(figsize=(8, 6))
plt.bar(cv_methods, mse_scores)
plt.xlabel("Cross-Validation Method")
plt.ylabel("Mean Squared Error")
plt.title("Mean Squared Error for Different Cross-Validation Techniques")
plt.show()
```



ENSEMBLE METHODS

- Instead of relying on one model (like Decision Tree), we use multiple models together and have them vote or collaborate.
- The idea: "The opinion of a group of people is more accurate than the opinion of one person."
- Benefits: Increases Accuracy, Reduces Overfitting, and Makes the model robust.

Basic Types

Bagging	Boosting	Stacking
▪ We create many copies of the same model. ▪ Each copy is trained on a different part of the data. ▪ Finally, we perform voting. ▪ The models run parallel (at the same time) ▪ The idea: Same model, Different data, Decision = Vote. ▪ A famous example: Random Forest.	▪ The models run sequentially (sequential). ▪ Each new model focuses on the errors made by the previous one ▪ Each model contributed to reducing errors. The final result became more accurate. ▪ The idea: Model after model Each one learns from the mistakes of the previous one. ▪ Examples: AdaBoost, Gradient Boosting, and XGBoost	▪ We use different models (SVM + Decision Tree + Logistic Regression). Each model gives its opinion. ▪ We take their results ▪ We give them to another model called Meta-learner to learn from their opinions and decides the correct answer. ▪ It's like: Asked 3 doctors. Then a fourth doctor made a decision based on Their words
Example: ✓ You have the same model (Decision Tree) but we repeat it several times: ✓ Model 1 is trained on a part of the data ✓ Model 2 is trained on a different part ✓ Model 3 is trained on a third part ✓ For the same client, the predictions: Model 1 said: Yes, Model 2 said: No, Model 3 said: Yes ✓ We take the majority opinion (Voting): Yes, No, Yes → Final Result: Yes	Example: ✓ Models are run consecutively and correct each other's mistakes ✓ Model 1: Incorrect in 10 samples ✓ Model 2: Focus on these 10 samples. Correct 7 of them. 3 still incorrect. ✓ Model 3: Focus on the remaining 3. Try to correct them.	Example: ✓ You have 3 models: Model 1 (Decision Tree), Model 2 (SVM), Model 3 (Logistic Regression), and they all try to predict: Will the customer buy the product or not? ✓ For the same client, the predictions were: Model 1 said: No, Model 2 said: Yes, Model 3 said: Yes. ✓ Instead of just voting. Let's look at a fourth model (Meta-Model). The Meta-Model sees: (No, Yes, Yes) → It learns that in this case, the correct answer is usually = Yes. So, the final prediction is: Yes.
▪ Bagging = Same model + different data. ▪ Reduce variance	▪ Boosting = Model after model + error correction ▪ Reduce errors	▪ Stacking = Different models + a model that governs them ▪ Best final decision

Iris Binary Classification

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
```

```
iris = datasets.load_iris()
```

```
df = pd.DataFrame(
    {'Sepal length': iris.data[:,0],
     'Sepal width': iris.data[:,1],
     'Petal length': iris.data[:,2],
     'Petal width': iris.data[:,3],
     'Species': iris.target})
df.head()
```

	Sepal length	Sepal width	Petal length	Petal width	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

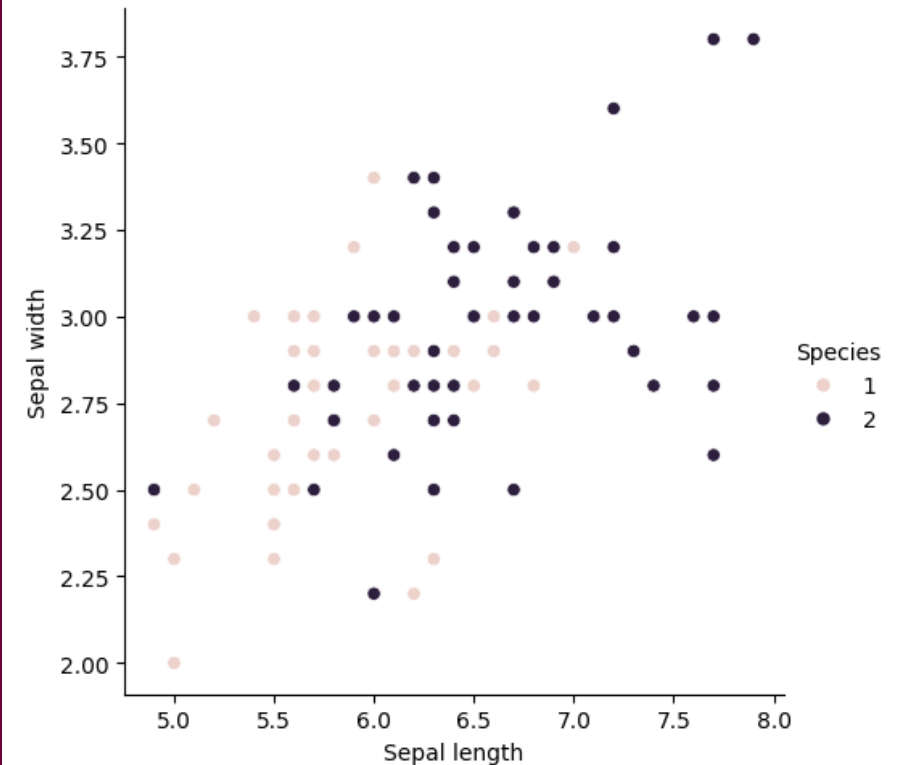
```
df = df[df['Species'] != 0]
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 100 entries, 50 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   Sepal length    100 non-null   float64
1   Sepal width     100 non-null   float64
2   Petal length    100 non-null   float64
3   Petal width     100 non-null   float64
4   Species         100 non-null   int64  
dtypes: float64(4), int64(1)
memory usage: 4.7 KB
```

```
sns.relplot(data = df, x = 'Sepal length', y = 'Sepal width', hue = 'Species')
```

```
<seaborn.axisgrid.FacetGrid at 0x1afb46bbb60>
```



```
from sklearn.model_selection import train_test_split
X = df[df.columns[:4]]
y = df[df.columns[-1]]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20 ,random_state=42)
```

*# `random\_state=42` --> It's a number we use to ensure randomness.
 # This means we make sure the results are the same every time you run the code.
 # It doesn't have to be 42; we can put any number.*

X_train

	Sepal length	Sepal width	Petal length	Petal width
105	7.6	3.0	6.6	2.1
138	6.0	3.0	4.8	1.8
76	6.8	2.8	4.8	1.4
92	5.8	2.6	4.0	1.2
119	6.0	2.2	5.0	1.5
...	...	...	...	...
110	6.5	3.2	5.1	2.0
121	5.6	2.8	4.9	2.0
64	5.6	2.9	3.6	1.3
142	5.8	2.7	5.1	1.9
101	5.8	2.7	5.1	1.9

80 rows × 4 columns

```
from sklearn.ensemble import RandomForestClassifier

# n_estimators=10 --> Create 10 decision trees.
# Each tree will be trained on a different piece of data.
# And in the end, they will all vote.
classifier=RandomForestClassifier(n_estimators=100)

classifier.fit(X_train,y_train)
```

Iris Binary Classification Using Random Forests

```
y_pred=classifier.predict(X_test)
```

```
from sklearn.metrics import classification_report ,confusion_matrix,accuracy_score
print('Accuracy:',accuracy_score(y_test,y_pred))
```

Accuracy: 0.95

Iris Binary Classification Using Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier()
gb_model.fit(X_train, y_train)

y_pred_gb = gb_model.predict(X_test)

from sklearn.metrics import accuracy_score

print("Gradient Boosting Accuracy:", accuracy_score(y_test, y_pred_gb))
```

Gradient Boosting Accuracy: 0.95

Iris Binary Classification Using Stacking

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

# Base Models
estimators = [
    ('dt', DecisionTreeClassifier()),
    ('svc', SVC(probability=True))
]

# Meta Model
stack_model = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)

stack_model.fit(X_train, y_train)

y_pred_stack = stack_model.predict(X_test)

print("Stacking Accuracy:", accuracy_score(y_test, y_pred_stack))

Stacking Accuracy: 0.95
```


Task

- Create a synthetic dataset that follows a **non-linear quadratic relationship** with added noise to simulate real-world data.
- Using **NumPy**, generate the dataset as follows:
 1. **Generate input feature:** Create 100 evenly spaced values between 0 and 6
 2. **Generate target variable:** Use the following mathematical relationship:
$$y = 2x^2 - 3x + \text{noise}$$
 3. **Add Gaussian noise with:** mean = 0 and standard deviation = 0.7
- Plot X vs y to understand the pattern.
- Train a **Linear Regression model** and observe: How well it fits the nonlinear data and whether it underfits or not.
- Apply Polynomial Regression with different degrees:
 1. **Degree = 3** (check if it captures the curve better).
 2. **Degree = 40** (observe what happens when the model becomes too complex and check for **overfitting**)
- Apply the following models: **Ridge Regression, Lasso Regression, Elastic Net Regression.**
- For each model, compute: Mean Squared Error (MSE) , R^2 Score



Task

- Generate a synthetic dataset and evaluate a **Linear Regression model** using different **cross-validation techniques** to compare model performance.
- Use NumPy to create a noisy linear dataset:

```
import numpy as np
import matplotlib.pyplot as plt

# Ensure reproducibility
np.random.seed(42)

# Generate input feature
X = np.linspace(0, 8, 120).reshape(-1, 1)

# Generate target variable with noise
y = 3 * X + np.random.normal(0, 1.5, X.shape[0]).reshape(-1, 1)

# Visualize data
plt.scatter(X, y)
plt.title("Generated Dataset")
plt.show()
```
- Evaluate a **Linear Regression model** using the following cross-validation methods:
 1. 4-Fold Cross Validation
 2. 6-Fold Cross Validation
 3. 8-Fold Cross Validation
 4. Leave-One-Out Cross Validation
- For each method: Split the data using the selected CV technique, Train a Linear Regression model, Predict on test folds, and Compute **Mean Squared Error (MSE)**
- Store the average MSE for each method, Print results in a clear format, and plot a bar chart comparing all methods



Task

- Apply Iris Multi Classification Using Random Forests.
- Apply Iris Multi Classification Using Gradient Boosting.
- Apply Iris Multi Classification Using Stacking.



Thank You

